Room 1 Burp Suite - Repeater

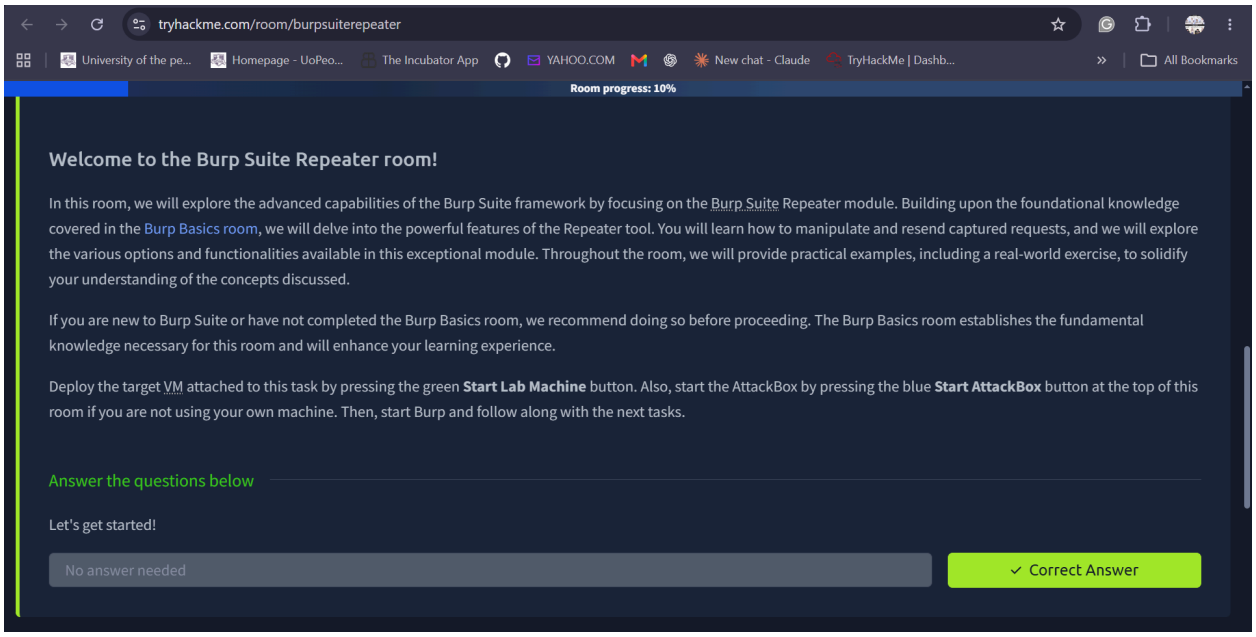
Platform: TryHackMe
Objective: Learn to use Burp Suite's Repeater module to intercept, modify, and resend HTTP requests to manually test endpoints and application behaviour.

Task 1: Introduction

Notes:

> This room builds on Burp Basics and focuses on **Repeater**, a module that lets you take a captured request, edit it, and resend it as many times as needed. This is far more efficient than manually intercepting every single request, since Repeater lets you quickly iterate on one request and compare responses.

Screenshot:



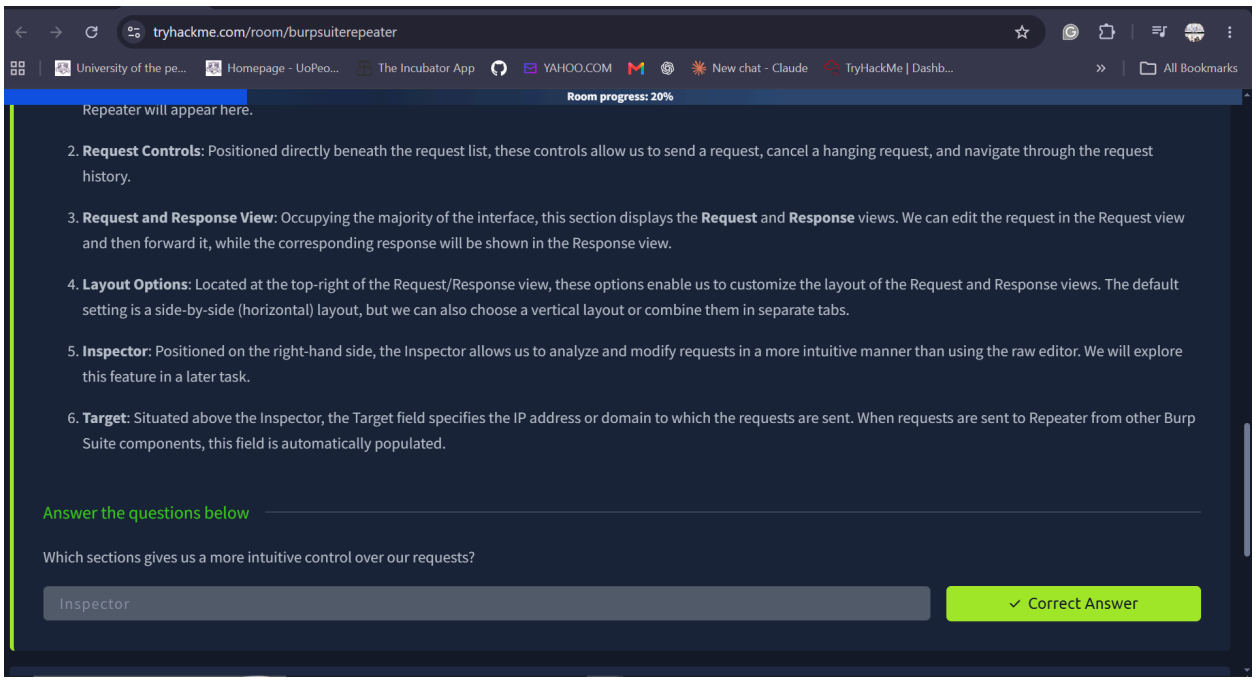
Task 2: What is Repeater?

Notes:

> Repeater lets you take requests captured in the Proxy and modify/resend them, or build requests from scratch, like using curl. The interface has six parts:
>
> - **Request list:** shows all Repeater requests/tabs

- **Request controls:** send, cancel, navigate history
- **Request/Response view:** edit the request, see the response
- **Layout options:** horizontal, vertical, or tabbed view
- **Inspector:** a more intuitive, structured way to view/edit requests
- **Target:** shows which host/IP the request is going to

Screenshot:



Task 3 & 4: Message Analysis Toolbar

Notes:

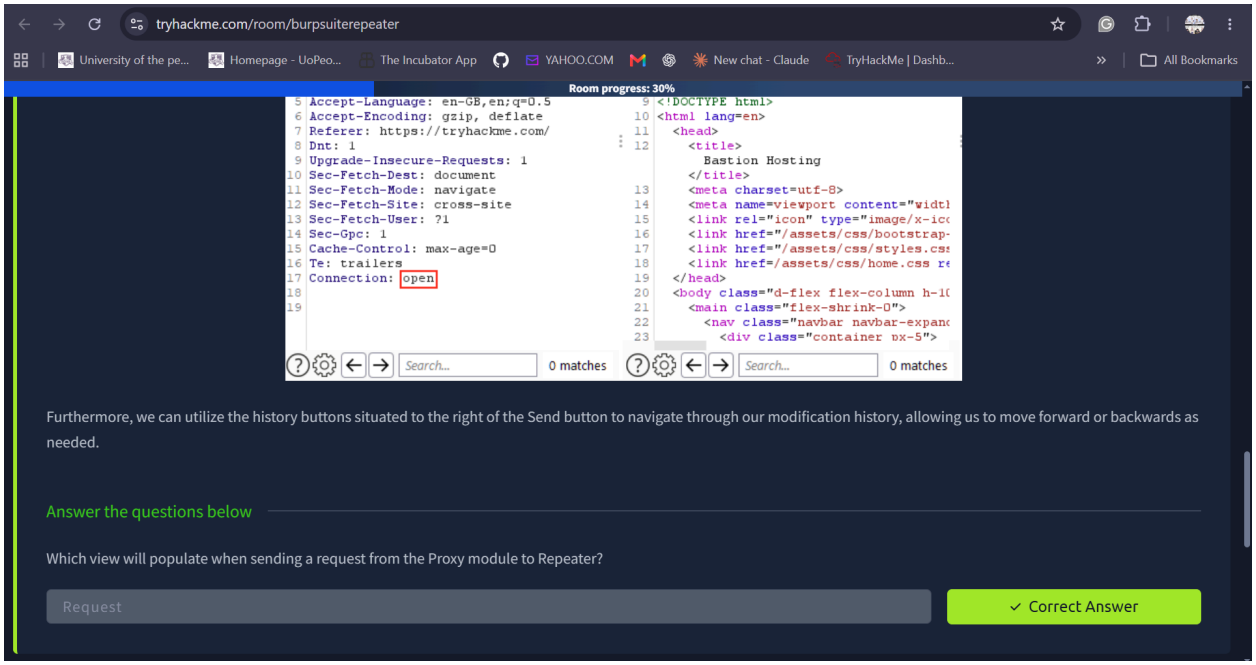
Above the response panel, Repeater offers four display options:

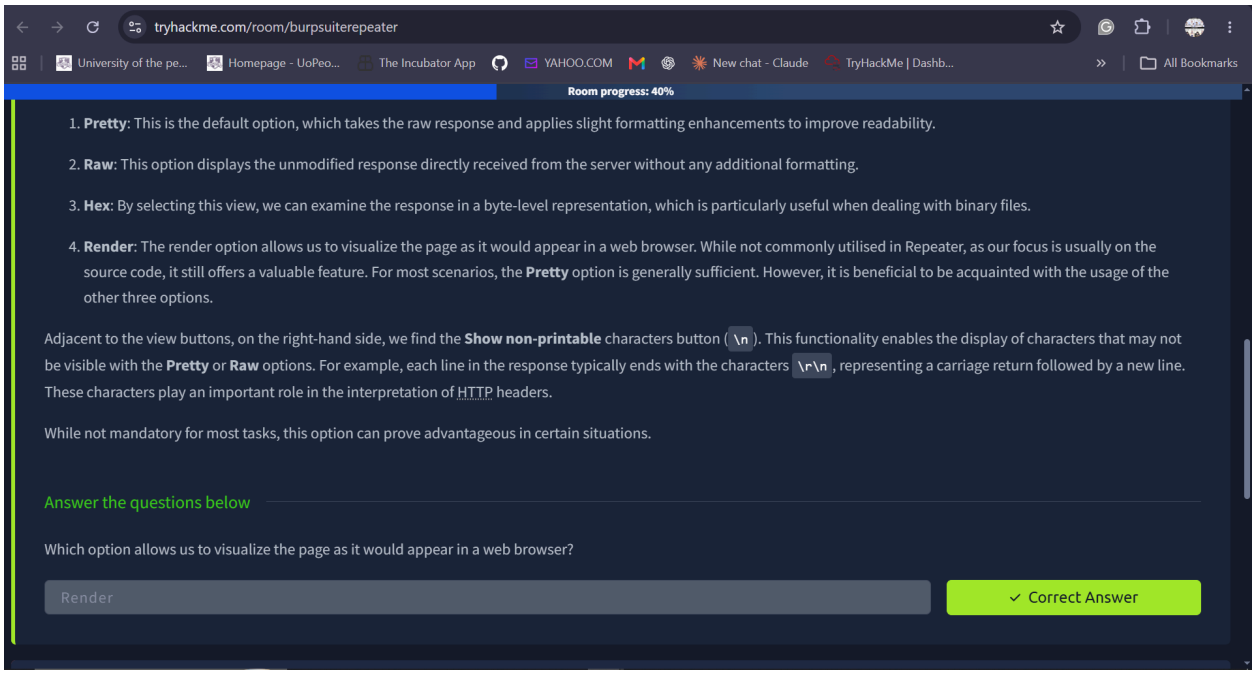
View	Purpose
Pretty	Default; lightly formats the raw response for readability
Raw	The unmodified response exactly as received
Hex	Byte-level view, useful for binary data

Render	Shows the page as it would appear in a browser

There's also a **Show non-printable characters (\n)** toggle, which reveals characters like \r\n (carriage return + newline) that normally stay hidden but matter for interpreting HTTP headers correctly.

Screenshots:





Task 5: Inspector

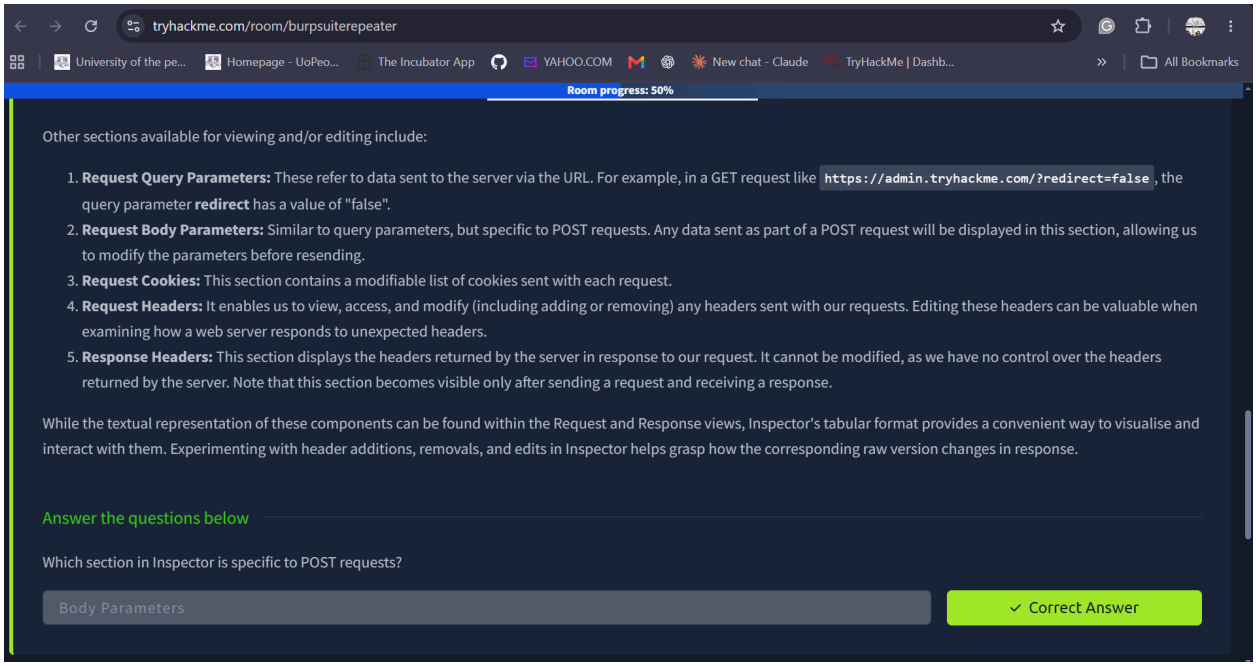
Notes:

Inspector gives a structured, tabular breakdown of a request/response, available in both Proxy and Repeater. Editable sections include:

- **Request Attributes** (method, path, protocol — e.g. switching GET to POST, or HTTP/1 to HTTP/2)
- **Request Query Parameters** (data sent via the URL, e.g. ?redirect=false)
- **Request Body Parameters** (POST data)
- **Request Cookies**
- **Request Headers**
- **Response Headers** (view only, since the server controls these)

Editing through the Inspector updates the raw request automatically, which is a faster way to understand how changes affect the underlying HTTP message.

Screenshot:



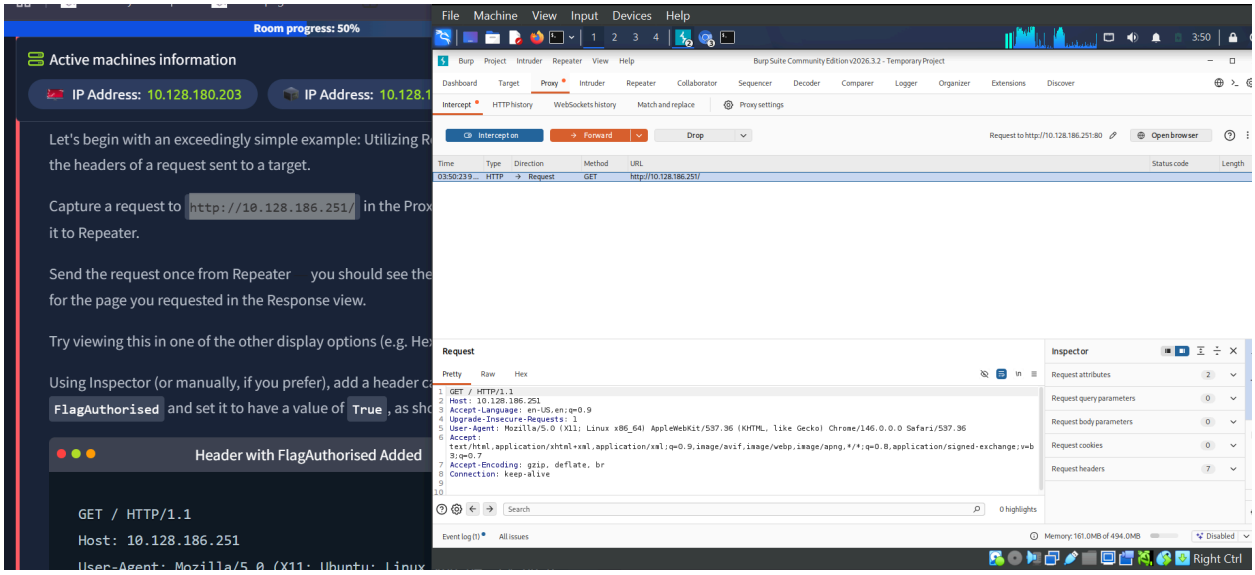
Task 6: Practical Example - Adding a Header

Notes:

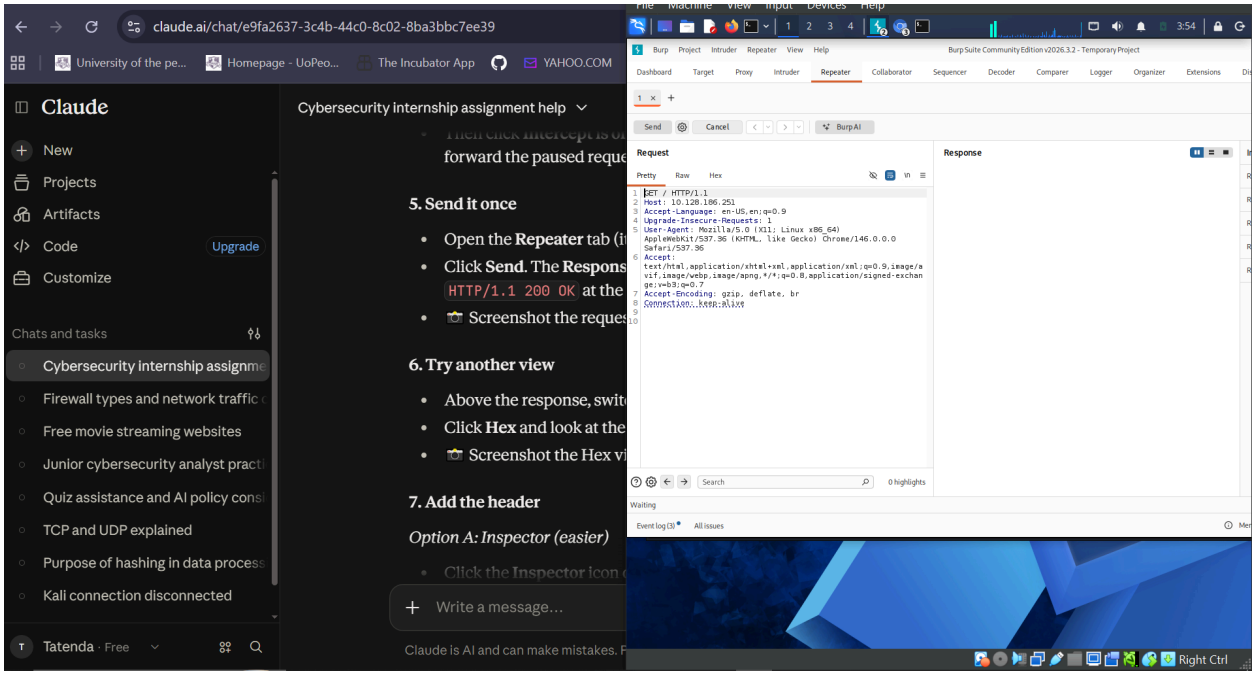
Repeater is ideal for repetitive testing with small changes, such as SQLi testing, WAF bypass attempts, or tweaking form parameters. In this example, I captured a request to the target's homepage, sent it to Repeater, and viewed the response in Pretty and Hex views. I then added a custom header, FlagAuthorised: True, using Inspector, and resent the request. The server's response changed to reveal a flag, showing it was checking for this specific client-supplied header to grant access, a clear access control flaw, since the client should never be trusted to self-declare its own authorisation.

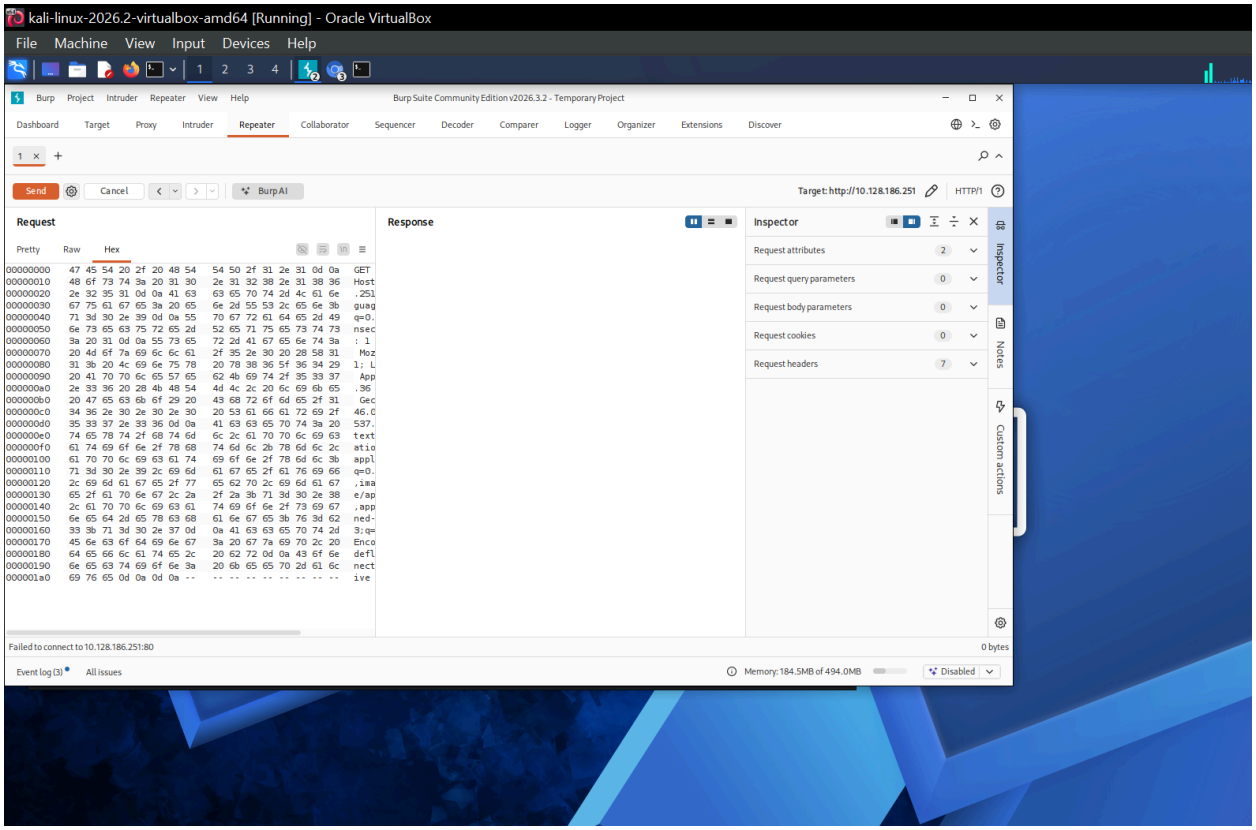
Steps performed:

1. Captured GET / in Proxy, sent to Repeater

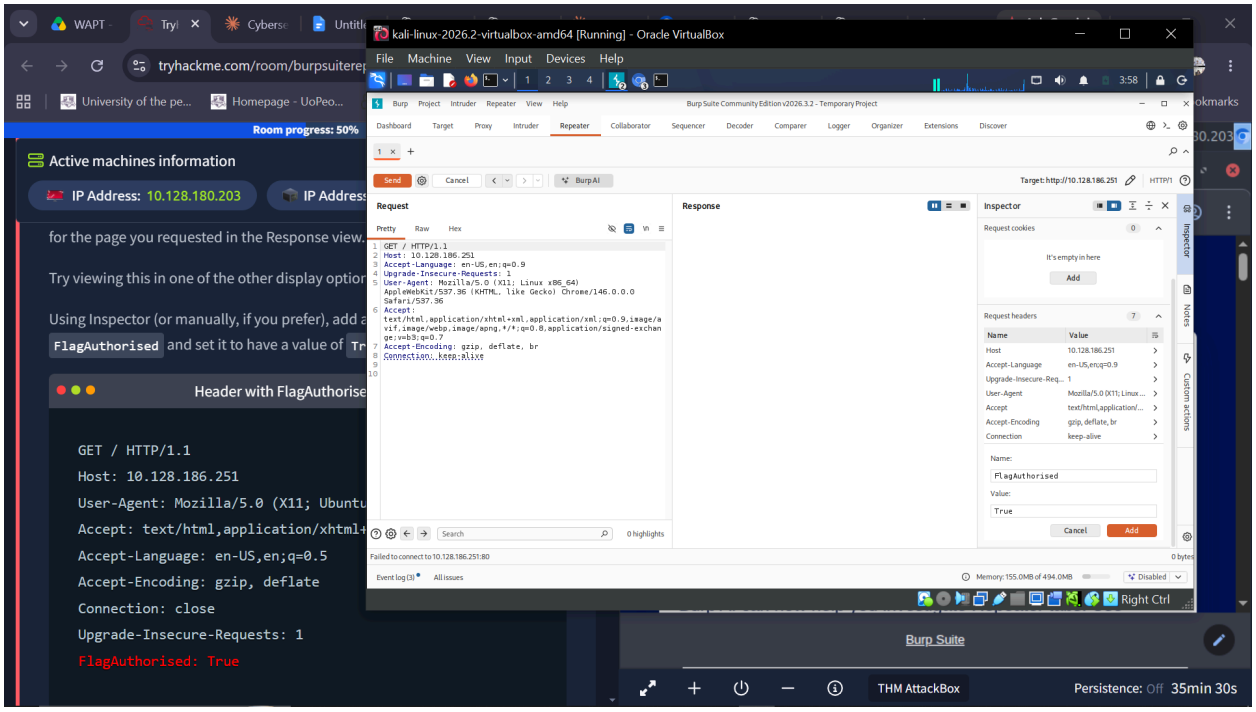


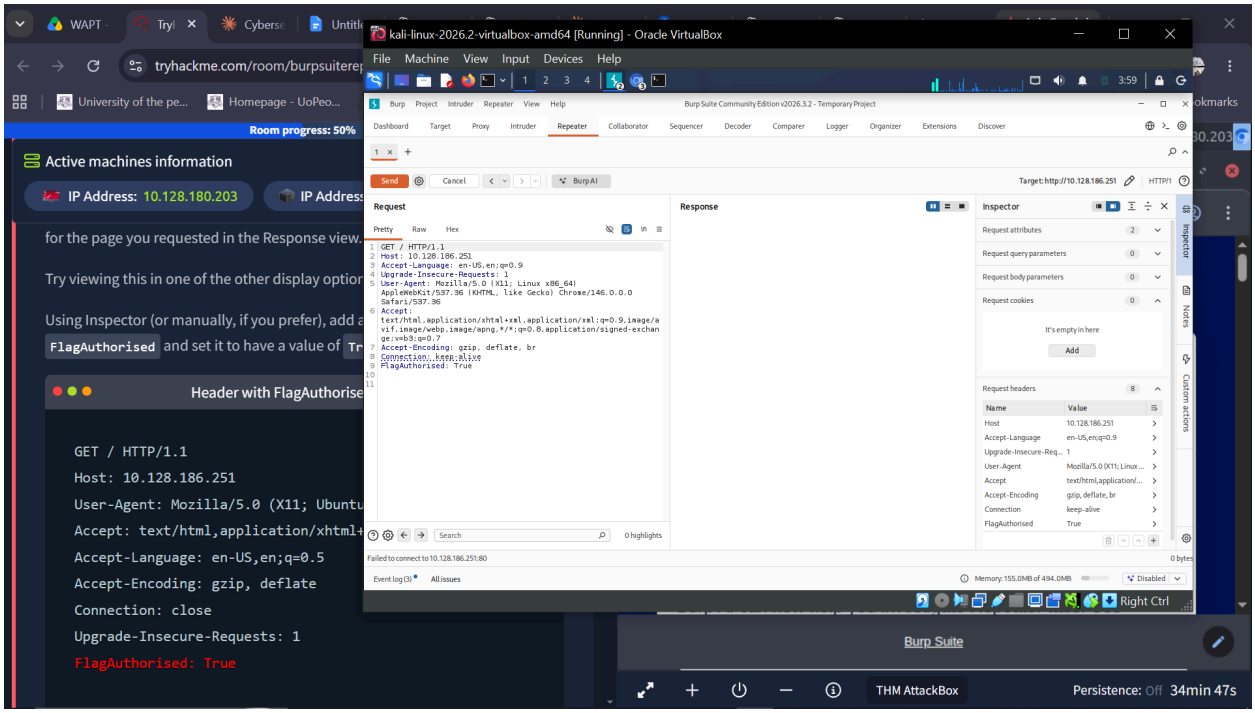
2. Sent once, viewed HTML in Pretty, then viewed in Hex



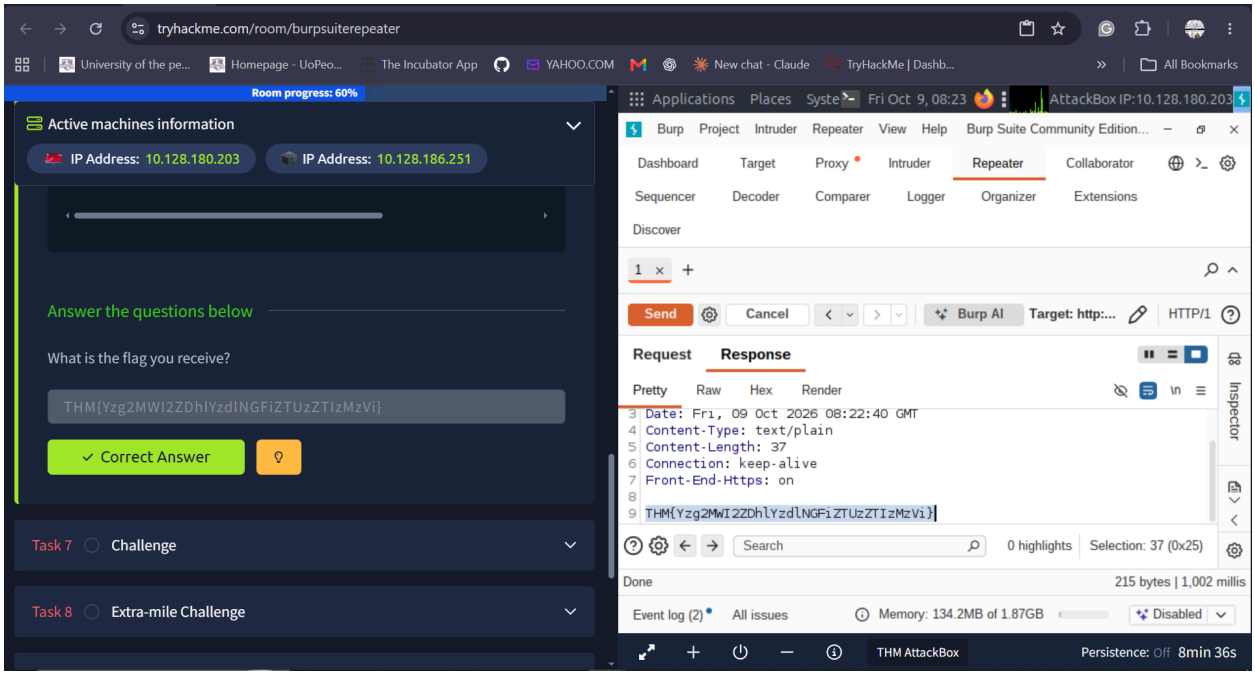


3. Added header FlagAuthorised: True via Inspector





4. Sent again, flag appeared in response



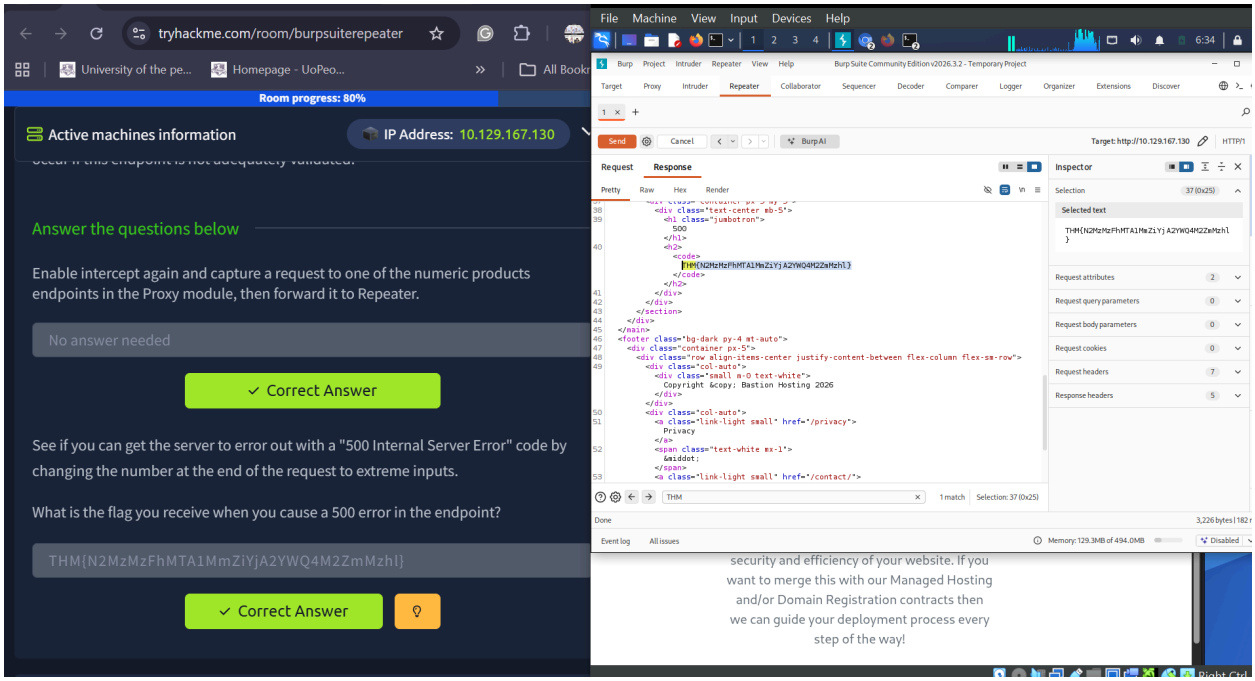
Screenshots:

Task 7: Challenge - Causing a 500 Error

Notes:

The /products/ page redirects to numeric endpoints like /products/3. The challenge was to test whether this endpoint validates its input properly. I captured a request to a numeric endpoint and sent it to Repeater, then tried extreme inputs in place of the valid number (very large numbers, decimals, negative numbers, non-numeric text) until I triggered a 500 Internal Server Error. The flag appeared in the error response. This showed the endpoint doesn't properly validate that the input is a valid integer before using it; an unhandled error case like this can also leak information through stack traces or verbose error messages.

Screenshots:



Task 8: Extra-Mile Challenge - Union SQL Injection

Notes:

This challenge targeted the id parameter of /about/ID. I captured a request to /about/2, sent it to Repeater, and added a single apostrophe to the end of the ID (/about/2'). This caused a 500 Internal Server Error, and the response body leaked the full underlying SQL query, confirming the vulnerability and revealing the table name (people) and that the query selected 5 columns.

I then used a UNION ALL SELECT query against information_schema.columns to enumerate the table's actual column names. On my instance, a plain query returned empty results, which I diagnosed as a schema-matching issue; adding AND table_schema=database() to restrict the query to the application's own database schema resolved it and returned the full column list: id, firstName, lastName, pfpLink, role, shortRole, bio, notes.

I identified the CEO's id as 1 and crafted a final query selecting the notes column for that user, which revealed the flag.

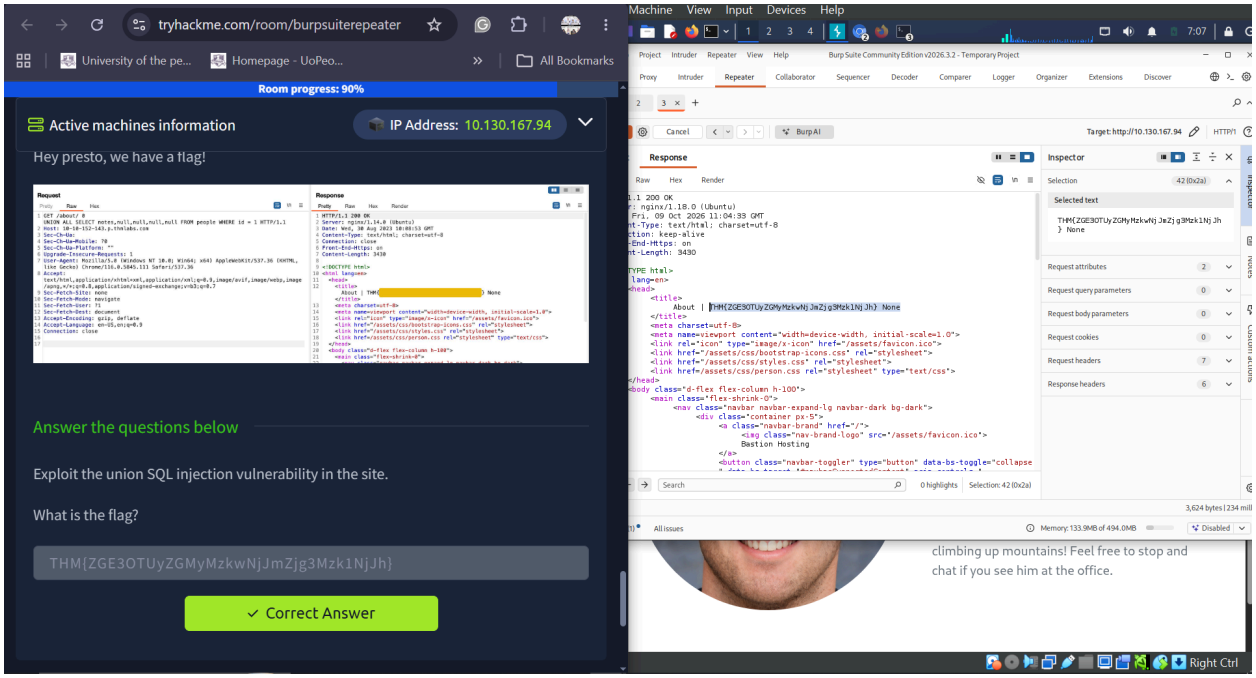
Final query used:

GET /about/0 UNION ALL SELECT notes,null,null,null,null FROM people WHERE id=1 HTTP/1.1

Problem-solving note:

My first information_schema.columns query returned "None" instead of real data, even though no SQL error occurred. I diagnosed this step by step: confirmed the injection mechanics worked using a test value ('TESTVALUE'), ruled out SQLite (sqlite_master caused a 500 error, meaning it didn't exist), then confirmed the database was MySQL-compatible by successfully listing tables with information_schema.tables WHERE table_schema=database(). This pinpointed that my original column query was missing the schema restriction, not that the vulnerability itself was wrong.

Screenshots:

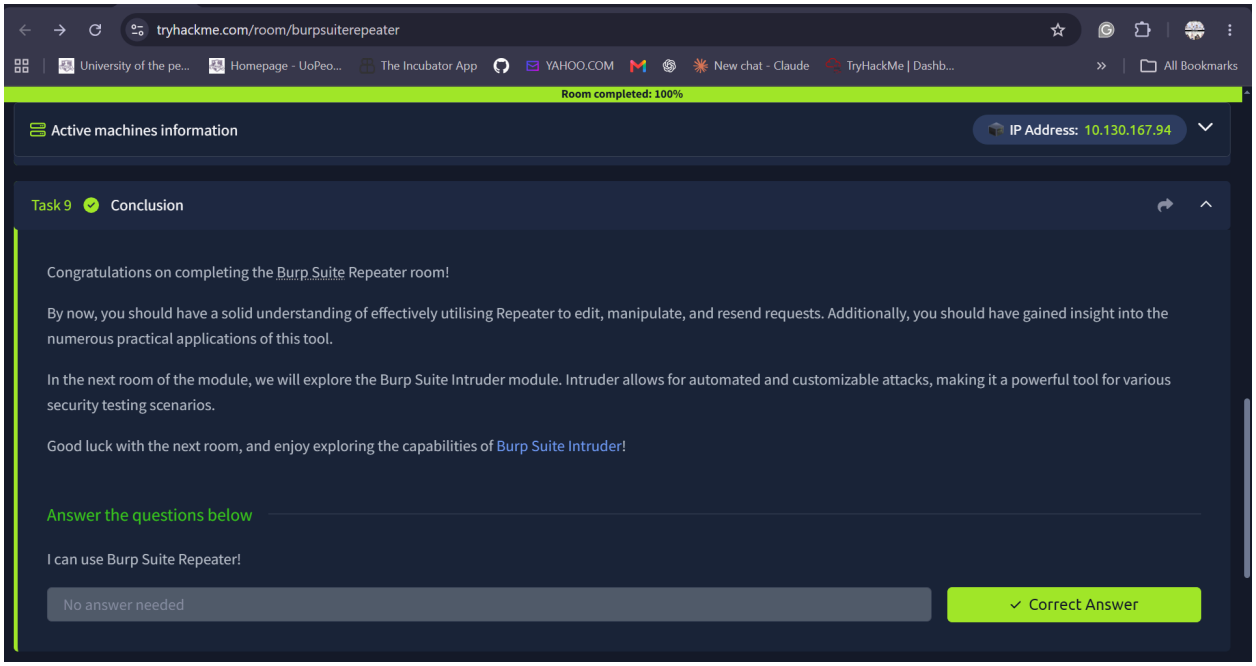


Task 9: Conclusion

Notes:

This room covered Repeater's core workflow: capture a request, send it to Repeater, edit it (headers, parameters, body, or path), and resend to compare responses. I practised this through a header-based access control bypass, an input-validation 500 error, and a full Union SQL Injection exploitation chain. The next room in the module covers Burp Suite Intruder, for automated/repeated attacks.

Screenshot:



Tools used

Burp Suite Community Edition (Proxy, Repeater, Inspector), Firefox/Burp's built-in browser, Kali Linux, OpenVPN

Problem-solving note (connectivity, spans the whole room)

> I had repeated connectivity failures throughout this room (Connection refused, No route to host, empty Firefox responses) because my Kali VM was never connected to TryHackMe's private lab network, it only had VirtualBox's default NAT network. I resolved this by locating TryHackMe's VPN settings under **Manage Account → VM and VPN Settings**, downloading the OpenVPN configuration file directly from Kali's own browser (to avoid a host-to-VM file transfer issue), and connecting with sudo openvpn. I confirmed the fix by checking for an active tun0 interface and successfully pinging the target machine.

Security observations to include

- **Client-controlled headers should never grant authorisation.** The FlagAuthorised header example showed that trusting client-supplied values for access decisions is a serious flaw.
- **Unvalidated numeric input** caused an unhandled server crash (500 error), which can leak internal details and signals a lack of input validation.

- **Verbose SQL error messages** directly exposed the query structure, table name, and column names, making exploitation significantly easier. Production systems should never return raw database errors to users.
- **Remediation:** validate and sanitise all input server-side, use parameterised queries instead of string concatenation, never make authorisation decisions based on client-supplied data, and disable detailed error messages in production.

Final "What I learned" paragraph

> This room taught me how to use Burp Repeater as a focused tool for manually testing how an application responds to modified requests. I practised capturing requests, editing headers and parameters through both the raw view and Inspector, and systematically narrowing down a SQL injection vulnerability by combining error messages with schema enumeration. The most valuable lesson was learning to diagnose problems methodically. When my enumeration queries kept failing silently, I isolated each possible cause (database type, schema, syntax) one at a time rather than guessing randomly, which is a core skill for real penetration testing. I also strengthened my understanding of network troubleshooting by finally resolving the VPN connectivity issues that had affected my work since the Recon section.

Completion link:

https://tryhackme.com/room/burpsuiterepeater?utm_campaign=social_share&utm_medium=social&utm_content=share-completed-room&utm_source=copy&sharerId=6a298268d8b6b164348fab72